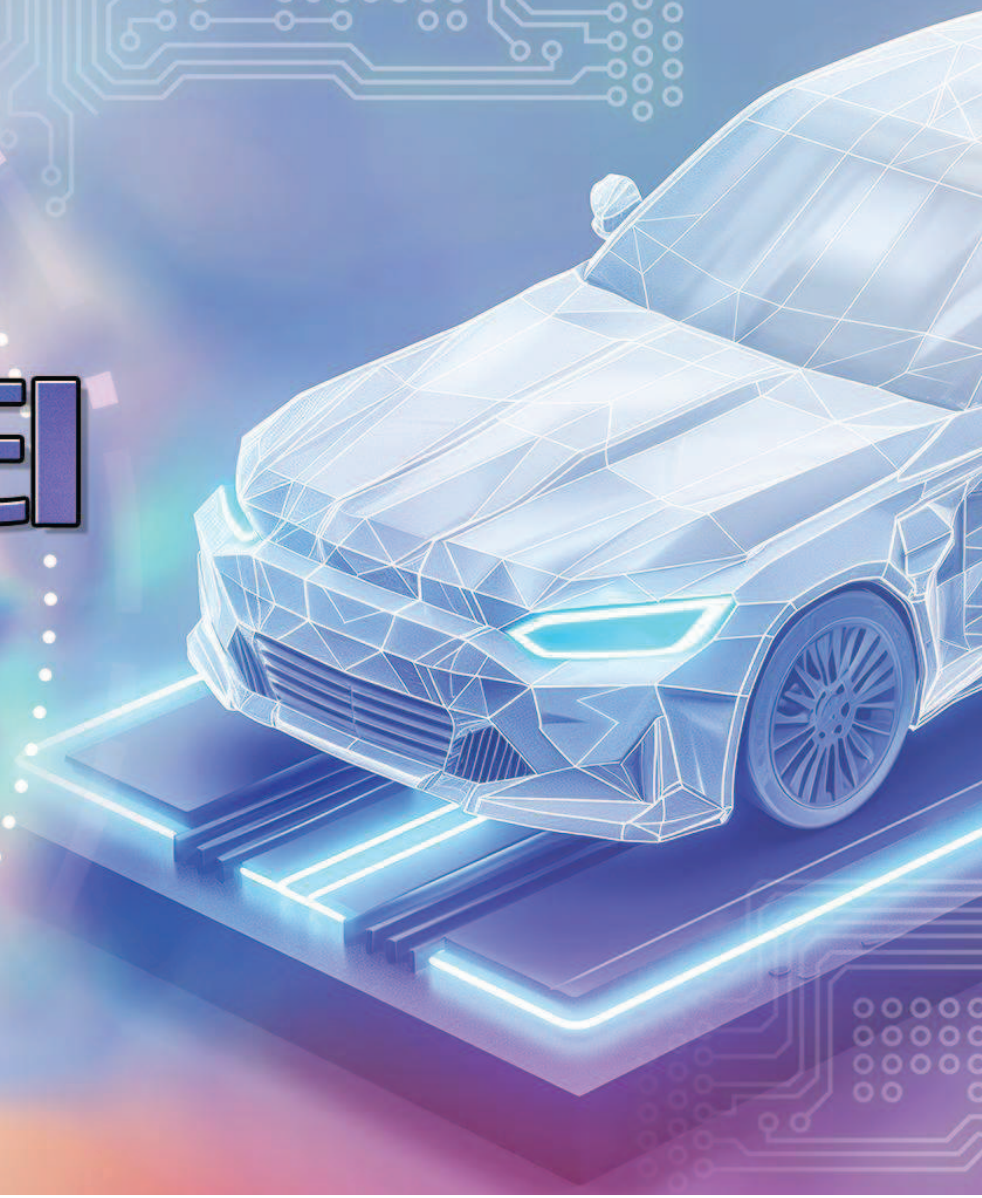
전북지역혁신플랫폼
Jeonbuk Regional Innovation Platform

JST

지능형모빌리티 설계

5주차 ROS2 프로그래밍 #3

2차시 ROS1 패키지 ROS2로 변경하기



* 본 영상은 무단 배포할 수 없습니다.

학습내용

1 Git과 Github

2 ROS1 → ROS2 변환

학습목표

- Git을 사용할 수 있다.
- ROS1 패키지를 ROS2로 변경할 수 있다.

전북지역혁신플랫폼
Jeonbuk Regional Innovation Platform

Git & Github 기초

Git이란?

왜 사용해야 하는가?

HW_지능형모빌리티_v1.cpp
HW_지능형모빌리티_v2.cpp
HW_지능형모빌리티_v2(최종).cpp
HW_지능형모빌리티_v2(진짜최종).cpp
...
HW_지능형모빌리티_final.cpp
HW_지능형모빌리티_final_final.cpp

Git이란?

왜 사용해야 하는가? Version 관리

- 내가 원하는 시점, 원하는 버전에 자유롭게 이동 가능



Git이란?

왜 사용해야 하는가? Version 관리

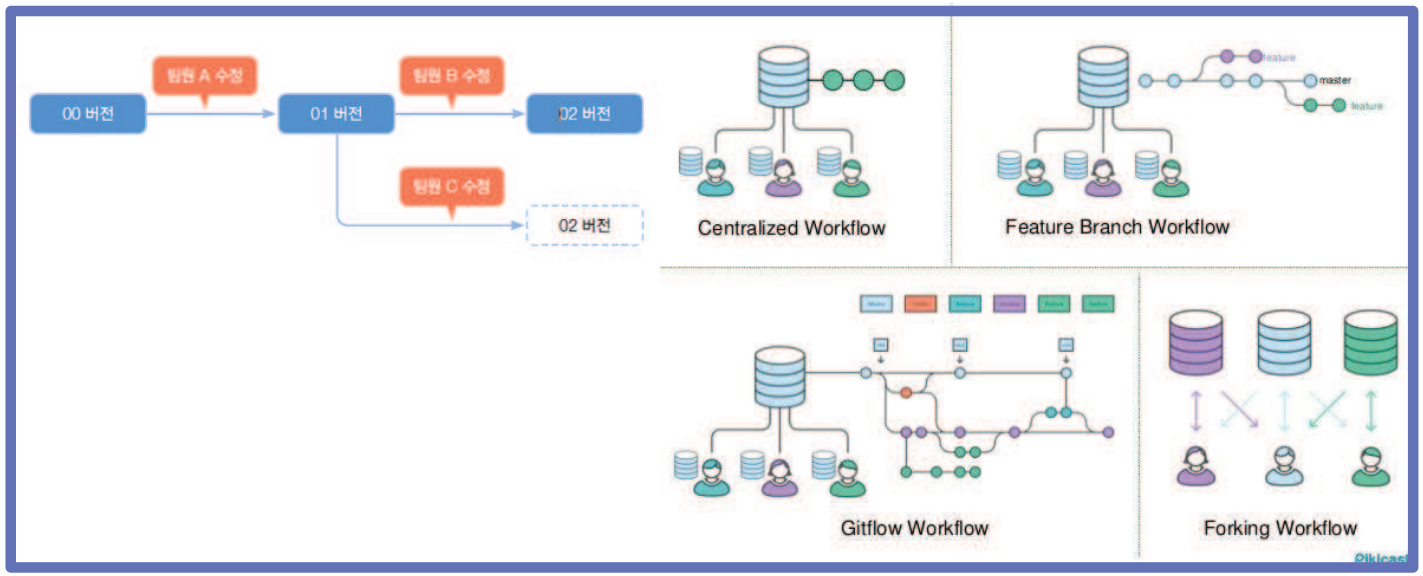
- 여러 명이 같이 하는 Project라면?

```
Project_지능형모빌리티_v1.cpp
Project_지능형모빌리티_v2.cpp
Project_지능형모빌리티_v2(최종).cpp
Project_지능형모빌리티_v2(진짜최종).cpp
Project_지능형모빌리티김북대_v2(최종).cpp
Project_지능형모빌리티팀1_합본.cpp
...
Project_지능형모빌리티_final.cpp
Project_지능형모빌리티_final_final.cpp
Project_팀1_합본_최종.cpp
```

Git이란?

왜 사용해야 하는가? Version 관리

- 여러 명이 같이 하는 Project라면?
- 따로 조금씩 작업하다가 원할 때 합치기 + 백업도 가능
- 기존 버전과 현재 버전을 비교하여 바뀌어 있는 부분 확인 가능



Git이란?

요약

- 코딩할 때(코딩에만 한정되어 있지 않음)
  단순히 ctrl + z를 눌러 이전 상태로 되돌리는 것이 아니라,
  원하는 시점마다 깃발을 꽂고(버전을 만들고)
  이들 간에 자유롭게 돌아다닐 수 있다.

- 내가 만든 버전 뿐만 아니라
  동료가 만든 버전으로 이동할 수 있고,
  동료와 내 버전을 비교해서
  최신본으로 코드를 업데이트 할 수 있다.

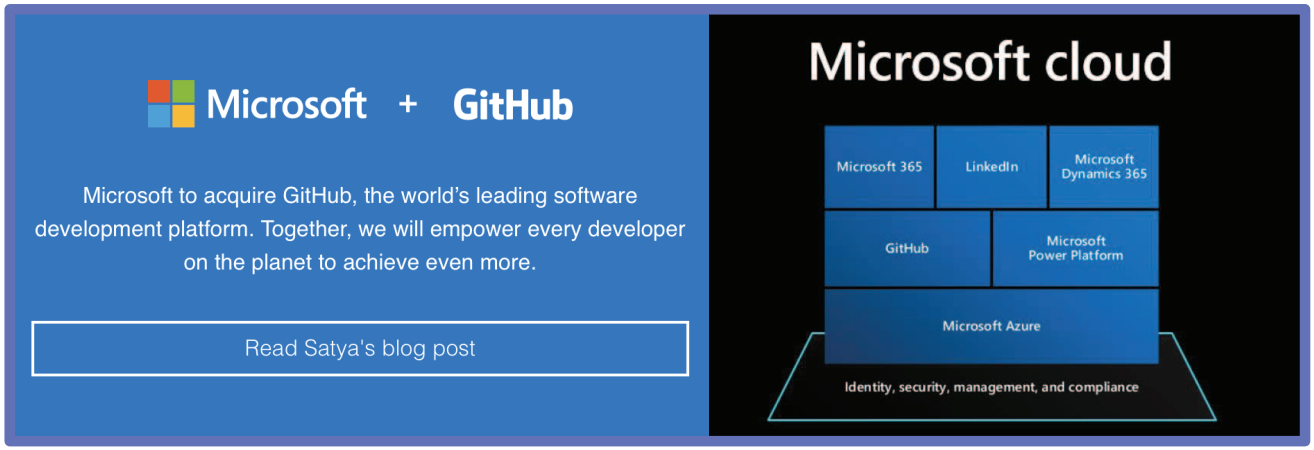
Git이란?

요약

- Git(버전이 관리되는 작업공간)은 어디에나 저장 가능함

USB PC 특정 서버 클리 우드 웹

(GitHub, GitLab, BitButcket)



전북지역혁신플랫폼
Jeonbuk Regional Innovation Platform

ROS1 → ROS2

ROS1 오픈 소스

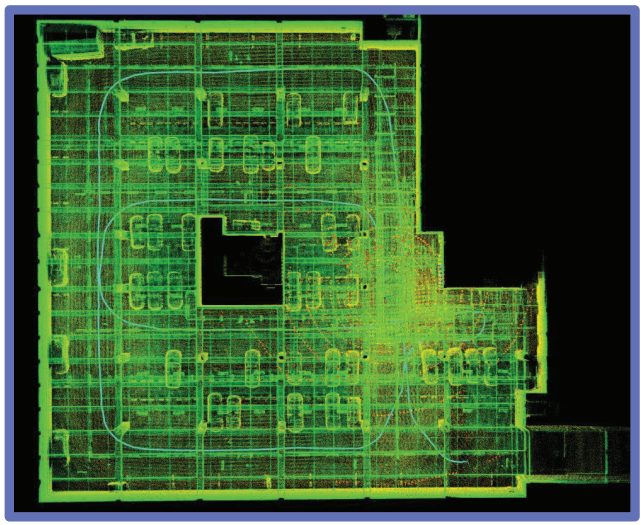
특징

- 대다수의 오픈 소스가 ROS1 버전으로 작성되어 있음
  - ROS1은 Ubuntu 20.04 버전까지만 사용할 수 있음
  - ROS1은 유지 관리 보수만 진행 중
  - ROS1의 한계점
- 기존의 오픈 소스를 원활히 활용하기 위해서는 ROS2로의 변경이 필요

Faster-LIO

LiDAR SLAM

- Faster-LIO[1]
  - LiDAR와 IMU를 사용해 로봇 위치를 추정하고 지도를 작성하는 알고리즘
  - 효율적이고 빠르게 위치 추정 및 지도 관리가 가능함
- Github: https://github.com/gaoxiang12/faster-lio
  - ROS1으로 작성되어 있음
- 본 자료는 ROS1에서 ROS2로의 주요 번경점만을 담고 있음
  - ROS2 버전(https://github.com/GangGeon10/Faster-LIO-ROS2)



Faster-LIO 실행 결과

출처 [1] BAI, Chunge, et al. Faster-LIO: Lightweight tightly coupled LiDAR-inertial odometry using parallel sparse incremental voxels. IEEE Robotics and Automation Letters, 2022, 7.2: 4861-4868.

ROS2 주요 변경점

ROS 사용

- ROS1에서 사용하는 NodeHandel은 ROS2에서는 사용 불가

```
// ROS1
#include <ros/ros.h>
ros::NodeHandle &nh
```

```
// ROS2
#include <rclcpp/rclcpp.hpp>
std::shared_ptr<rclcpp::Node>& node
```

- Class 선언 시 아래와 같이 Node임을 선언

```
class LaserMapping : public rclcpp::Node {
          //
};
```

msg type

ROS2 주요 변경점

msg

- ROS2에서는 msg 경로가 변경되며, 사용할 때도 `msg::`가 추가됨
- Header 파일 Include 시 대문자는 사용하지 않으며,
  중간에 대문자 포함 시 "_"으로 연결

```
// ROS1
#include <sensor_msgs/PointCloud2.h>
#include <nav_msgs/Path.h>
sensor_msgs::PointCloud2
```



```
// ROS2
#include <sensor_msgs/msg/point_cloud2.hpp>
#include <nav_msgs/msg/path.hpp>
sensor_msgs::msg::PointCloud2
```

ROS2 주요 변경점

Subscriber & Publisher

- Topic을 사용하기 위한 Subscriber와 Publisher
- 선언

```
// ROS1
ros::Subscriber sub_pcl_;
ros::Publisher pub_laser_cloud_world_;
```



```
// ROS2
rclcpp::Subscription<sensor_msgs::msg::PointCloud2>::SharedPtr sub_pcl_;
rclcpp::Publisher<sensor_msgs::msg::PointCloud2>::SharedPtr pub_laser_cloud_world_;
```

ROS2 주요 변경점

Subscriber & Publisher

정의

```
// ROS1
sub_pcl_ = nh.subscribe<sensor_msgs::PointCloud2>(
           lidar_topic, 200000, [this](const sensor_msgs::PointCloud2::ConstPtr &msg) { StandardPCLCallBack(msg); });
pub_laser_cloud_world_ = nh.advertise<sensor_msgs::PointCloud2>("/cloud_registered", 100000);
```



```
// ROS2
sub_pcl_ = this->create_subscription<sensor_msgs::msg::PointCloud2>(
           lidar_topic, 200000, [this](const sensor_msgs::msg::PointCloud2::SharedPtr msg)
{ StandardPCLCallBack(msg); });
pub_laser_cloud_world_ =
this->create_publisher<sensor_msgs::msg::PointCloud2>(“/cloud_registered", 100000);
```

ROS2 주요 변경점

Time

- ROS1의 header.stamp의 toSec(), fromSec() 기능 없음
- 해당 기능을 수행하도록 코드 변경

```
//ROS1
msg->header.stamp.toSec()
msg->header.stamp = ros::Time().fromSec(timediff_lidar_wrt_imu_ +
msg_in->header.stamp.toSec());
```



```
//ROS2
(double)msg->header.stamp.sec + (double)1e-9* msg->header.stamp.nanosec
rclcpp::Time t(static_cast<uint64_t>((timediff_lidar_wrt_imu_ + ((double)
msg_in->header.stamp.sec +
(double)1e-9* msg_in->header.stamp.nanosec))*1e9 ));
msg->header.stamp = t;
```

ROS2 주요 변경점

Parameter

declare_parameter와 get_parameter를 이용해 Parameter 설정

```
// ROS1
nh.param<bool>("path_save_en", path_save_en_, true);
nh.param<std::vector<double>>("mapping/extrinsic_R", extrinR_, std::vector<double>());
```



```
// ROS2
this->declare_parameter("path_save_en",  rclcpp::PARAMETER_BOOL);
this->declare_parameter(("mapping/extrinsic_R", rclcpp::PARAMETER_DOUBLE_ARRAY);
if (!this->get_parameter("path_save_en", path_save_en_)) {
        RCLCPP_WARN(this->get_logger(), "Parameter path_save_en not found");
    }
if (!this->get_parameter("mapping/extrinsic_R", extrinR_)) {
        RCLCPP_WARN(this->get_logger(), "Parameter extrinR_ not found");
    }
```

파라미터 자료형

ROS2 주요 변경점

Yaml 파일

- Node 이름과 ros_parameters가 추가되어야 하며 파라미터는 들여쓰기 되어야 함
- 정의된 파라미터의 자료형과 동일한 자료형으로 사용해야 함

```
# ROS2
laserMapping:            # For ROS2 , node name
    ros__parameters:     # For ROS2
        common:
            lid_topic:  "/ouster/points"
            imu_topic:   "/ouster/imu"
~~~
~~~
        mapping:
                        ~~~
                        ~~~
            extrinsic_R: [1.0, 0.0, 0.0,
                          0.0, 1.0, 0.0,
                          0.0, 0.0, 1.0] # For ROS2, double
```

ROS2 주요 변경점

본 예제는 python 형태로 작성: mapping_ouster64.launch.py

```python
import os
from ament_index_python.packages import get_package_share_directory
from launch import LaunchDescription
from launch_ros.actions import Node
import launch_ros.actions
from launch.substitutions import PathJoinSubstitution

def generate_launch_description():
    config_file = PathJoinSubstitution([get_package_share_directory('faster_lio'), 'config', 'ouster64.yaml'])
    rviz_config = PathJoinSubstitution([get_package_share_directory('faster_lio'), 'rviz_cfg', 'loam_livox.rviz'])

    faster_lio_node=Node(
        package='faster_lio',
        executable="run_mapping_online",
        output='screen',
        parameters=[ config_file
                   ])
    rviz_node = Node(
         package='rviz2',
         executable='rviz2',
         name='rviz2',
         arguments=['-d', rviz_config,
                    '--ros-args', '--log-level', 'warn'],
         parameters=[{'use_sim_time' : True}],
         output='screen')
    ld = LaunchDescription([launch_ros.actions.SetParameter(name='use_sim_time', value=True),])
    ld.add_action(faster_lio_node)
    ld.add_action(rviz_node)
    return ld
```

파라미터 파일

rviz 설정 파일

Faster-lio Node

rviz2 Node

Node 실행

ROS2 주요 변경점

의존성 설치

```
$ sudo apt-get install libgoogle-glog-dev
$ sudo apt-get install libeigen3-dev
$ sudo apt-get install libpcl-dev
$ sudo apt-get install libyaml-cpp-dev
```

Livox-SDK2

```
$ git clone https://github.com/Livox-SDK/Livox-SDK2.git
$ cd ./Livox-SDK2/
$ mkdir build
$ cd build
$ cmake .. && make -j
$ sudo make install
```

ROS2 주요 변경점

Build

CMakeLists.txt와 package.xml 변경 후 Build

```
$ cd ~/ros2_ws
$ colcon build --symlink-install
$ source install/setup.bash
```

ROS2 주요 변경점

실행

```
$ ros2 launch faster_lio mapping_ouster64.launch.py
$ ros2 bag play <ros2_bag_file> --clock
```